

TRACK A / AUDIT PRODUCT ENGINEERING

Repair the register

Make a small invoice application dependable, then add one useful improvement.

The situation

ClearLedger is a local application for a fictional service business. The owner needs a register they can trust before a busy week. The app runs and its starter tests pass, but six deliberate defects remain.

Your mission

- 1. Investigate.** Reproduce important problems, explain their impact and choose what to tackle first.
- 2. Repair.** Fix priority problems. Include a failing-before/passing-after regression check and an additional input case you designed. Record the actual results.
- 3. Improve.** Add a small improvement beyond the behaviours already required in `BUSINESS_RULES.md`. Explain the need, show it working and check it.
- 4. Hand over.** Give exact run/test commands, evidence and known limits in the shared handover template.

The owner has noticed

"The open-invoice view doesn't agree with the overview. Retrying an import sometimes moves the numbers again. The export and screen don't always agree. An import said complete, but some records weren't there."

These observations are starting points, not the full defect list. The intended behaviour is in `track-a/BUSINESS_RULES.md`. Create your own cases where useful.

Seven calendar days to submit. Four hours of work.

Choose one track. Include reading, choosing, setup, coding, tests and handover in the four-hour limit. You may split the work across sessions. AI is welcome; no paid model is required. You do not need to fix every defect.

Get it running

Extract the full assessment ZIP. In the **track-a** folder, use Python 3.10+ and a modern browser. The starter has no third-party packages, accounts or API keys.

```
python app.py
```

Open **http://127.0.0.1:8787**. Keep the terminal open; Ctrl+C stops the server. Use **py** on Windows or **python3** on macOS/Linux if needed. If the port is busy, use **python app.py --port 8790** and open that port.

Run checks and preserve existing data

```
python -m unittest discover -s tests -v
# Stop the server before restoring:
python restore_fixture.py --replace
python app.py
```

The five starter tests are smoke checks. The supplied register has **nine invoices, five payments and INR 3,698.19 outstanding**. Preserve its records, support new imports and verify a restart. Test any schema migration. See **fixtures/README.md** for exact records and setup. Resetting to the demo does not satisfy preservation.

What correct behaviour looks like

- 1. Identity matters.** Invoice identity is customer ID plus invoice number. An amount alone must never match a payment. Unknown references stay visible as unmatched.
- 2. Retries are safe.** Identical invoices or payments are skipped. Reused identities with changed details are rejected without overwriting records.
- 3. Rows fail independently.** A bad data row must not discard good rows. A bad header rejects the whole file. Report counts, line numbers and reasons.
- 4. Money agrees.** Preserve cents; show and export two decimal places. Open means positive balance; paid means zero or negative. Overpayments do not offset other invoices.
- 5. Feedback is truthful.** The browser shows actual import results and useful errors. It must never claim success after a failed request.

Read the detailed rules and public API before changing behaviour. Keep the documented routes and response fields; internal code can change. All records are fictional, all amounts are INR, and accounting expertise is not required.

Show your working

We value a carefully checked result and a clear account of its limits. A focused partial repair can outperform a larger, unverified rewrite.

Criterion	Weight	What we look for
Correctness and reliability	30%	Useful fixes that satisfy the rules without breaking other workflows
Verification	25%	Reproductions, regression checks and important edge cases
Investigation and prioritization	20%	A clear explanation of impact and what you tackled first
Useful improvement	15%	A justified, checked change beyond the required repairs
Handover and tool judgment	10%	Reproducible work, accurate limits and checked tool output

A suggested time budget

35 min read, run and investigate / **130 min** fix and test

40 min one improvement / **35 min** clean-run check and handover

Reallocate as needed. If repairs use the available time, describe your proposed improvement and its first test. An implemented, verified improvement receives more credit. Stop after four hours.

Your submission

Reply with an accessible download link to **YourName_TrackA.zip**, or a repository link, within **seven calendar days of the invitation**. Include code, tests and **HANDOVER.md**. Use the shared template; keep the handover under 500 words excluding commands.

Include dependencies, migration code and the original **fixtures/**. Exclude generated working databases, caches and credentials. Check link access; send a link instead of an attachment. No public repository, video, slide deck or deployment is required.

Using AI and getting help

Any model, search or documentation is welcome; free tools are enough. Work individually and credit outside code. Give two or three short examples of a tool suggestion, your decision and how you checked it. Full chat histories are not required.

Work independently within the requirements. Document assumptions and unresolved questions; no solution guidance will be provided. Contact the inviter only for inaccessible/missing materials or an accessibility adjustment. Shortlisted applicants will explain and adapt their work in a separate **15-20 minute discussion**, with AI allowed. See **START_HERE.md**.